

REST Routes in an Existing Backend

How does the server know what you want?

Agenda

1. Opening problem
2. Prior knowledge check
3. Core concepts
 - URL vs. verb
 - The JSON envelope
 - Content negotiation
 - Idempotence
4. Hands-on: add Makes & Categories routes
5. Reflection

Is this API broken?

```
DELETE /api/cars/7    → 200 OK  
DELETE /api/cars/7    → 404 Not Found
```

Same request. Different response.

Broken — or not?

True or False?

| "A REST API should use `/api/cars/delete/1` to delete a car."

Discuss with the person next to you.

URL = resource name | Verb = action

Verb	URL	Meaning
GET	/api/cars	List all cars
POST	/api/cars	Create a car
GET	/api/cars/1	Get car #1
PUT	/api/cars/1	Replace car #1
DELETE	/api/cars/1	Remove car #1

Why HTML routes look different

GET /cars/1/edit	← HTML: need the extra segment
POST /cars/1/edit	because forms only do GET and POST
GET /api/cars/1	← API: verb carries the intent
PUT /api/cars/1	no /edit or /delete needed
DELETE /api/cars/1	

Open `app/RouteProvider.php` — you can see both patterns side by side.

The JSON envelope

Every response has the same shape:

```
{  
  "meta": { "resource": "cars", "total": 12 },  
  "links": { "self": "/api/cars" },  
  "data": [ ... ]  
}
```

Key	Purpose
meta	Who, how many
links	Where to go next (HATEOAS)
data	The actual payload

HATEOAS — the server drives navigation

```
"links": {  
  "self": "/api/cars/1",  
  "collection": "/api/cars"  
}
```

A client that follows these links **never hardcodes a URL**.

If `/api/cars` becomes `/api/v2/cars` — clients that follow links keep working.

Content negotiation

Same data, different representation:

```
GET /cars      → CarController  → Twig  → HTML
GET /api/cars  → ApiController → JSON  → { ... }
```

Proper REST uses the `Accept` header.

Maestro uses the URL prefix — pragmatic, but worth knowing the difference.

Idempotence

Sending the same request N times = same server state as sending it once.

Method	Idempotent?
GET	✓ — read-only
PUT	✓ — replaces to a known state
DELETE	✓ — already gone; repeating is safe
POST	✗ — creates a new resource each time

Back to the opening question

```
DELETE /api/cars/7    → 200 OK      (car removed)
DELETE /api/cars/7    → 404        (car already gone)
```

The server state after both calls is identical.

Not broken. Idempotent.

Exercise 1 — Sequence Diagram

Trace `GET /api/cars` through the full stack.

```
Client → index.php → Kernel → Router → ApiController  
      → CarRepository → Database → back up → Response
```

Questions to answer:

- Where is the JSON envelope built?
- Where is it decided the response is JSON, not HTML?
- What happens if the repository returns an empty array?

Exercise 2 — Add Makes API Routes

`MakeRepository` exists. No API routes yet.

Your task:

1. Add `makes()` and `make()` to `ApiController.php`
2. Register the routes in `RouteProvider.php`
3. Test: `/api/makes` and `/api/makes/1`

Extension: nest cars inside `/api/makes/{id}`

Reflection

1. What is the difference between `POST /api/cars` and `PUT /api/cars/1` — in plain language?
2. Why don't API routes have `/edit` or `/delete` in them?
3. What do you understand differently now compared to the start of this lesson?

Homework

Exercise 3 — Add Categories API routes

(`GET /api/categories` and `GET /api/categories/{id}`)

Exercise 4 — Written reflection:

"A teammate suggests `/api/cars/search?q=mustang` . Is that RESTful?"

Take a position and argue it in 150–250 words.